PROJET 6
Optimisez la gestion & nettoyez les données du stock d'une boutique

ALLA Sofiane
Data Analyst
Avril 2025

SOMMAIRE :

I. Analyses exploratoires des données
II. Fusion ou consolidations des données
III. Analyses univariées du prix
IV. Analyses complémentaires / CA, quantités, stocks, taux de marge et correlations
V. Actions pour la suite
VI. Compétences apprises

I. Analyses exploratoires des données (1)

Comprendre le jeu de données

- *3 fichiers :*

➔ *erp.xlsx (product_id/onsale_web/price/stock_quantity/stock_status/purchase_price)*

➔ *web.xlsx (sku/virtual/downloadable/rating_count/average_rating/total_sales/tax_status/ tax_class/post_author/etc)*

➔ *liaison.xlsx (id_web/product_id)*

- *Nettoyages : gestion des prix nuls/négatifs, suppression des doublons, netttoyage de sku, etc*
- *Création de nouvelles variables (stock_status, roation_stock, CA, taux_marge, etc)*

- *Statistiques descriptives (moyenne, min, max, etc)*
- *Visualisation : boîte à moustache (price), diagramme en barre (CA et total_sales)*
- *Outliers (= données aberrantes) détécté via Z-Score et écart interquartile*

I. Analyses exploratoires des données (2)

```python
#Afficher les dimensions du dataset
df_erp.info()
#Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(df_erp.shape[1]))
#La nature des donnees dans chacune des colonnes
type(df_erp)
#Le nombre de valeurs (observations, caracteristiques) presentes dans chacune des colonnes
print("Le tableau comporte {} observation(s) ou article(s)".format(df_erp.shape[0]))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   product_id      825 non-null    int64
 1   onsale_web      825 non-null    int64
 2   price           825 non-null    float64
 3   stock_quantity  825 non-null    int64
 4   stock_status    825 non-null    object
 5   purchase_price  825 non-null    float64
dtypes: float64(2), int64(3), object(1)
memory usage: 38.8+ KB
Le tableau comporte 6 colonne(s)
Le tableau comporte 825 observation(s) ou article(s)
```

II. Fusion ou consolidations des données (1)

Nettoyage du jeu de données :

- *Attributs clés : product_id, sku, price, purchase_price, total_sales*
- *Problèmes : sku manquants, produits avec stock_quantity négatif dans erp.xlsx, doublons entre l'enregistrement d'image miniature et la description du produit dans web.erp, lignes complétements Null sauf 2 où les sku existent, etc*

- *Fusion de erp et liaison (merge), puis fusion avec web*
- *Problèmes : certains produits étaient présent dans un fichier mais absents dans un autre*

II. Fusion ou consolidations des données (2)

```
#Creation d'une colonne "stock_status_2"
#La valeur de cette deuxième colonne sera fonction de la valeur dans la colonne "stock_quantity"
#Si la valeur de la colonne "stock_quantity" est nulle renseigner "outofstock" sinon mettre "instock"
df_erp["stock_status_2"] = df_erp["stock_quantity"].apply(lambda x: "instock" if x > 0 else "outofstock" )
```

```
#Verifions que les 2 colonnes sont identiques:
#Les 2 colonnes sont strictement identiques si les valeurs de chaque ligne sont strictement identiques 2 a 2
#La comparaison de 2 colonnes peut se realiser simplement avec l'instruction ci-dessous:
df_erp["stock_status"] == df_erp["stock_status_2"]
'''Le resultat est l'affichage de True ou False pour chacune des lignes du dataset, c'est un bon debut, mais difficile à exploiter'''
```

```
#Si les colonnes ne sont absolument pas identiques ligne a ligne alors identifier la ligne en ecart
df_erp[df_erp["stock_status"] != df_erp["stock_status_2"]]
'''2 lignes erreurs -> 4 et 398'''
#Dans ce cas je vous ce lien pour apprendre a realiser des filtres dans Pandas : https://bitbucket.org/hrojas/learn-pandas/src/master/ (Lesson 3)
```

```
'2 lignes erreurs -> 4 et 398'
```

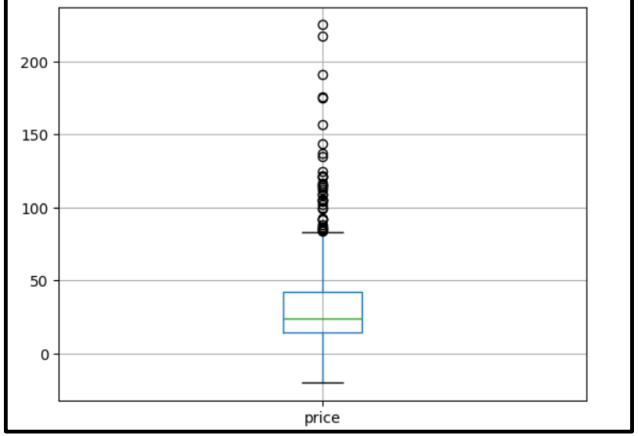
```
#Corriger la ou les donnees incoherentes
df_erp.loc[df_erp["product_id"]==4039, "stock_status"] = "instock"
df_erp.loc[df_erp["product_id"]==4885, "stock_status"] = "outofstock"
#Verification en utilisant le meme code que plus haut pour afficher les problemes
df_erp[df_erp["stock_status"] != df_erp["stock_status_2"]]
```

III. Analyses univariées du prix (1)

- *Outliers (= données aberrantes) détectés*
- *Analyse via boîte à moustache, Z-Score et écart interquartile*

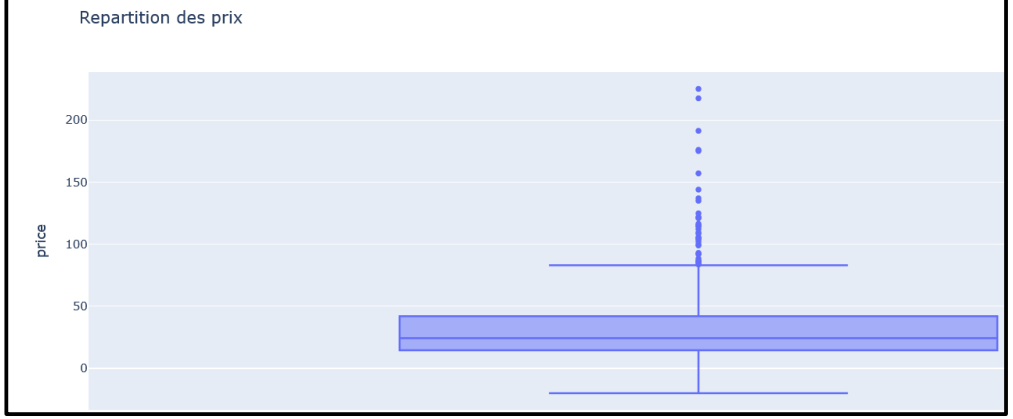
```
#Creation d'une Boite a moustache de la repartition des prix grace à Pandas
df_merge[["price"]].boxplot()
```

```
<Axes: >
```



Boîtes à moustache

```
#Autre methode avec plotly express (Q3*1.5*interquartile pour la limite des valeurs aberrantes)
px.box(data_frame=df_merge,y="price", title ="Repartition des prix")
```



III. Analyses univariées du prix (2)

Identification par le Z-index

```
#Calculer la moyenne du prix
moyenne = df_erp["price"].mean()
print(moyenne)
#Calculer l'ecart-type du prix
ecart_type = df_erp["price"].std()
print(ecart_type)
#Calculer le Z-score (=moyenne/ecart-type)
Z_score = moyenne/ecart_type
print(Z_score)
```

```
32.187696969696965
26.712076775505285
1.2049866897362609
```

```
#Quel est le seuil prix dont z-score est superieur à 3?
'''(chaque_prix-moyenne)/ecart_type=Z_score -> faire avec max et min
A partir de 102.3, les valeurs sont aberrantes'''
df_outliers = df_erp[df_erp["price"].apply(lambda x : (abs(x-moyenne)/ecart_type) > 3)]
df_outliers["price"].min()
```

```
114.0
```

Identification par l'intervalle interquartile

```
#Utilisation de la fonction describe de Pandas pour l'etude des mesures de dispersions
df_erp["price"].describe()
```

	price
count	825.000000
mean	32.187697
std	26.712077
min	-20.000000
25%	14.500000
50%	24.300000
75%	42.000000
max	225.000000

dtype: float64

```
#Definissez un seuil pour les articles "outliers" en prix
'''Formule du seuil = Q3+(Q3-Q1)*1.5'''
df_describe=df_erp["price"].describe()
seuil = df_describe.loc["75%"] + (df_describe.loc["75%"] - df_describe.loc["25%"])*1.5
print(seuil)
```

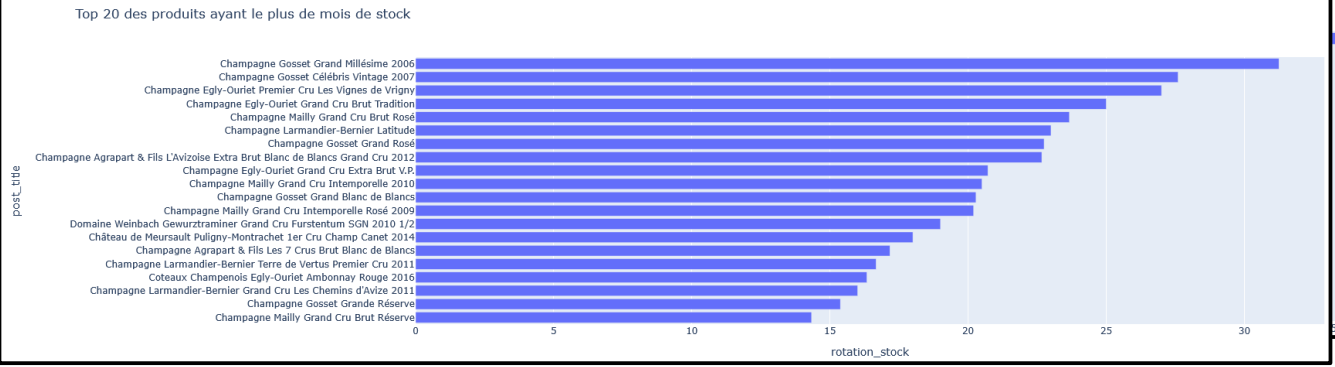
```
83.25
```

IV. Analyses complémentaires
CA, quantités, stocks, taux de marge et correlations (1)

- *Top 20 des produits par CA et ventes*
- *Taux de marge par type de produit*
- *Corrélation : price vs total_sales, stock, etc*

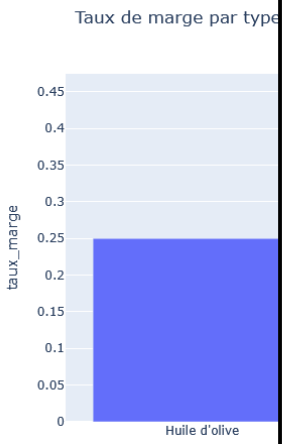
```
#Effectuer le tri dans l'ordre decroissant du CA du dataset df_merge
df
#Effectuer le tri dans l'ordre decroissant de quantites vendues du dataset df_merge
df
df
df
px.
```

```
#Import de numpy
import numpy as np
#Creation de la colonne Rotation de stock (quantite/nombre de vente)
df_merge["rotation_stock"] = df_merge["stock_quantity"]/df_merge["total_sales"]
#Remplacement des "inf" par 0
df_merge["rotation_stock"] = df_merge["rotation_stock"].replace([np.inf],0)
#Effectuer le tri dans l'ordre decroissant du nombre de mois de stock dans le dataset df_merge
df_merge.sort_values("rotation_stock", ascending=False, inplace=True)
#Graphique en barre du flop 20 des produits qui ont le plus de mois de stock
px.bar(data_frame=df_merge[:20][::-1], y="post_title", x="rotation_stock", orientation="h", title="Top 20 des produits ayant le plus de mois de stock")
```



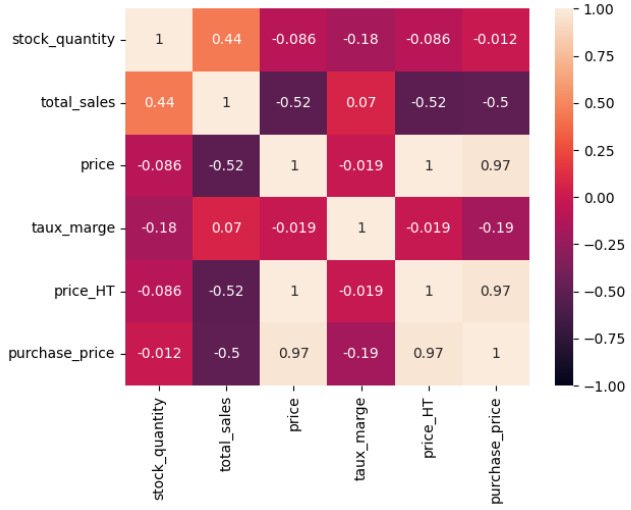
IV. Analyses complémentaires
CA, quantités, stocks, taux de marge et correlations (2)

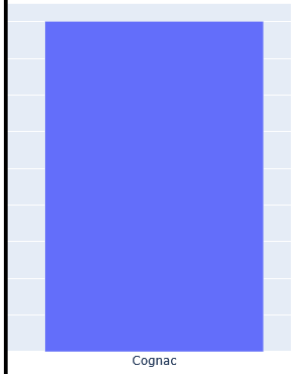
```
#Creation d'un dataframe avec le taux
df_tauxmargemoyen=df_tauxpositif.group
#Affichage dans un graphique du taux d
df_tauxmargemoyen.sort_values("taux_ma
px.bar(data_frame=df_tauxmargemoyen, y
```




```
#Importation de Seaborn
import seaborn as sns
#Creation d'un heatmap de correlation avec les variables stock, sales et price
#On peut egalement creer un mask pour n'afficher qu'une demi heatmap
sns.heatmap(df_merge[["stock_quantity", "total_sales", "price", "taux_marge", "price_HT", "purchase_price"]].corr(),vmin=-1,vmax=1, annot=True)
'''Analyse quantitative ici, pas qualitative'''
```

```
'Analyse quantitative ici, pas qualitative'
```





V. Actions pour la suite

- Supprimer produits non vendus ou avec des stocks très élevé
- Identifier les produits à forte valeur de stock ou de marge
- Analyser les marges par fournisseur/produit (info manquantes)
- Définir des seuils d’alerte de stock minimum pour éviter des ruptures
- Stratégie de déstockage pour les invendus (promotions, liquidation, etc)

➔ Accentuer les efforts sur les produits de luxe (loi de Pareto 20/80)

➔ Pas d’info de croissance (rajouter l’évolution des ventes en fonction du temps par exemples)

VI. Point sur les compétences apprises

- Compétences apprise :

- Maîtrise des outils comme pandas, plotly ou seaborn
- Fusion de bases hétérogènes
- Analyses statistiques avancées

- Difficultés :

- Nettoyage des sku
- Gestion des données manquantes
- Identifiants non appariés

- Tâches nécessitants le plus d’entraînement :

- Nettoyage de données complexes
- Visualisation des données avec plotly et seaborn